

RAPPELS : NOTIONS DE BASES DU LANGAGE C++

Enseignante : Pélagie HOUNGUE, PhD

PLAN

- ◉ Introduction
- ◉ Quelques spécificités du langage C++
- ◉ Le C++ et la programmation orientée objets
- ◉ Structure d'un programme C++
- ◉ Mon premier programme en C++
- ◉ Etapes de l'exécution d'un programme C++
- ◉ Compilation séparée
- ◉ Variables/types/mots clés/opérateurs
- ◉ Fonctions de saisie et d'affichage de données
- ◉ Les structures de contrôle
- ◉ Les instructions de branchement inconditionnel
- ◉ Transtypage
- ◉ Installation et prise en main de Dev-C++ ou Cod::Blocks
- ◉ Travaux pratiques

INTRODUCTION

◎ Langage de programmation

- Donner des instructions à l'ordinateur afin qu'il exécute des actions
- Automatiser des tâches

◎ Exemples d'application

- le navigateur web Google Chrome, qui permet de consulter des sites web ;
- l'explorateur de fichiers, qui permet de gérer les fichiers sur son ordinateur ;
- l'application de traitement de texte Microsoft Word, qui permet de rédiger lettres et documents.

◎ Le langage C

- Né dans les laboratoires Bell en 1972
- Langage impératif
- Utilisé dans de nombreux domaines
 - Développement de systèmes, de logiciels système
 - Ingénierie, base de données, gestionnaire de communication, outils bureautiques.

INTRODUCTION (2)

⊙ Langage de programmation en C++

- Créé en 1980
- Un des langages les plus célèbres au monde
- Très utilisé, notamment dans le secteur des jeux vidéo qui apprécie ses performances et ses potentialités
- Descendant du langage C (C avec les classes)

⊙ Caractéristiques du langage C++

- Normalisé par la norme ANSI/ISO en 1998
- Structuré et modulaire
 - Programme séparé en fichiers
 - Fichier séparé en fonctions
 - Fonction séparée en blocs
- Portable (de nombreux environnement/compilateurs disponibles)
- Extension du langage C, pour la programmation objet.

⊙ Le C++ propose de nouvelles fonctionnalités

- la Programmation Orientée Objet (POO)
 - Rend très puissant le langage, qui permet de programmer avec une approche différente du langage C.

QUELQUES SPÉCIFICITÉS DU LANGAGE C++

- ◉ Nouvelle forme de **commentaire** (en fin de ligne),
- ◉ Plus **grande liberté** dans l'**emplacement des déclarations**,
- ◉ **Notion de référence** facilitant la mise en œuvre de la transmission d'arguments par adresse,
- ◉ **Surdéfinition des fonctions** permettant l'attribution d'un même nom à différentes fonctions, la reconnaissance de la fonction réellement appelée se faisant d'après le type et le nombre d'arguments figurant dans l'appel (on parle parfois de signature),
- ◉ Nouveaux opérateurs de **gestion dynamique** de la mémoire : new, delete

QUELQUES SPÉCIFICITÉS DU LANGAGE C++ (2)

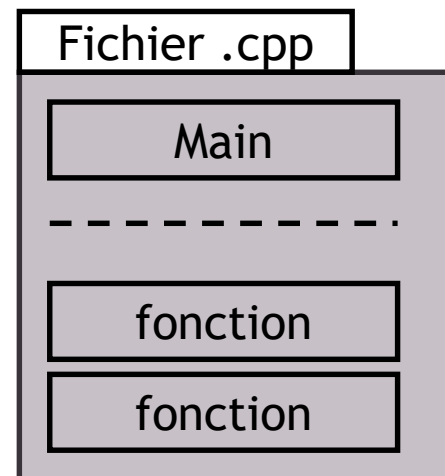
- ⊙ Possibilité de définir des **fonctions " en ligne"** (inline), ce qui accroît la vitesse d'exécution, sans perdre pour autant le formalisme des fonctions
- ⊙ Les **espaces de noms (namespace)** pouvant permettre d'utiliser des variables de même nom, définies dans un espace spécifique
 - Un namespace, est une zone de déclaration d'identificateurs permettant au compilateur de résoudre les conflits de noms.
- ⊙ Utilisation du **type bool** ayant pour valeur vrai ou faux
- ⊙ Les **nouveaux opérateurs de cast (transtyage)** permettant de réaliser des conversions explicites à l'aide d'un opérateur de "cast« (**static_cast**, **reinterpret_cast**, **const_cast**, **dynamic_cast**).

LE C++ ET LA PROGRAMMATION ORIENTÉE OBJETS

- ◉ La POO est en fait une évolution de l'approche modulaire. Elle lui apporte principalement trois aspects primordiaux:
 - **L'encapsulation**
 - technique permettant de réunir des variables et des fonctions au sein d'une même entité nommée classe et de les protéger à travers la définition de niveau de visibilité.
 - **L'héritage**
 - cette technique permet de définir une hiérarchie de classes. Chaque classe fille hérite des méthodes et des données de ses "pères".
 - **Le polymorphisme**
 - Faculté attribuée à un objet d'être une instance de plusieurs classes.
 - Cette propriété est très utile pour la création d'ensemble regroupant des objets de classes différentes.

STRUCTURE D'UN PROGRAMME C++

- ◉ Chaque fichier se présente comme une suite de fonctions.
- ◉ Chaque fonction a un rôle et peut appeler d'autres fonctions pour effectuer certaines actions.
- ◉ Une fonction unique: la fonction *main*
 - Fonction principale



MON PREMIER PROGRAMME

```
#include <iostream>
int main(){
std::cout << "Hello world!" << std::endl;
return 0;
}
```

```
#include <iostream>
using namespace std;
int main(){
cout << "Hello world!" << endl;
return 0;
}
```

QUELQUES EXPLICATIONS

- ◉ `#include <iostream>`

- « *Input Output Stream* »

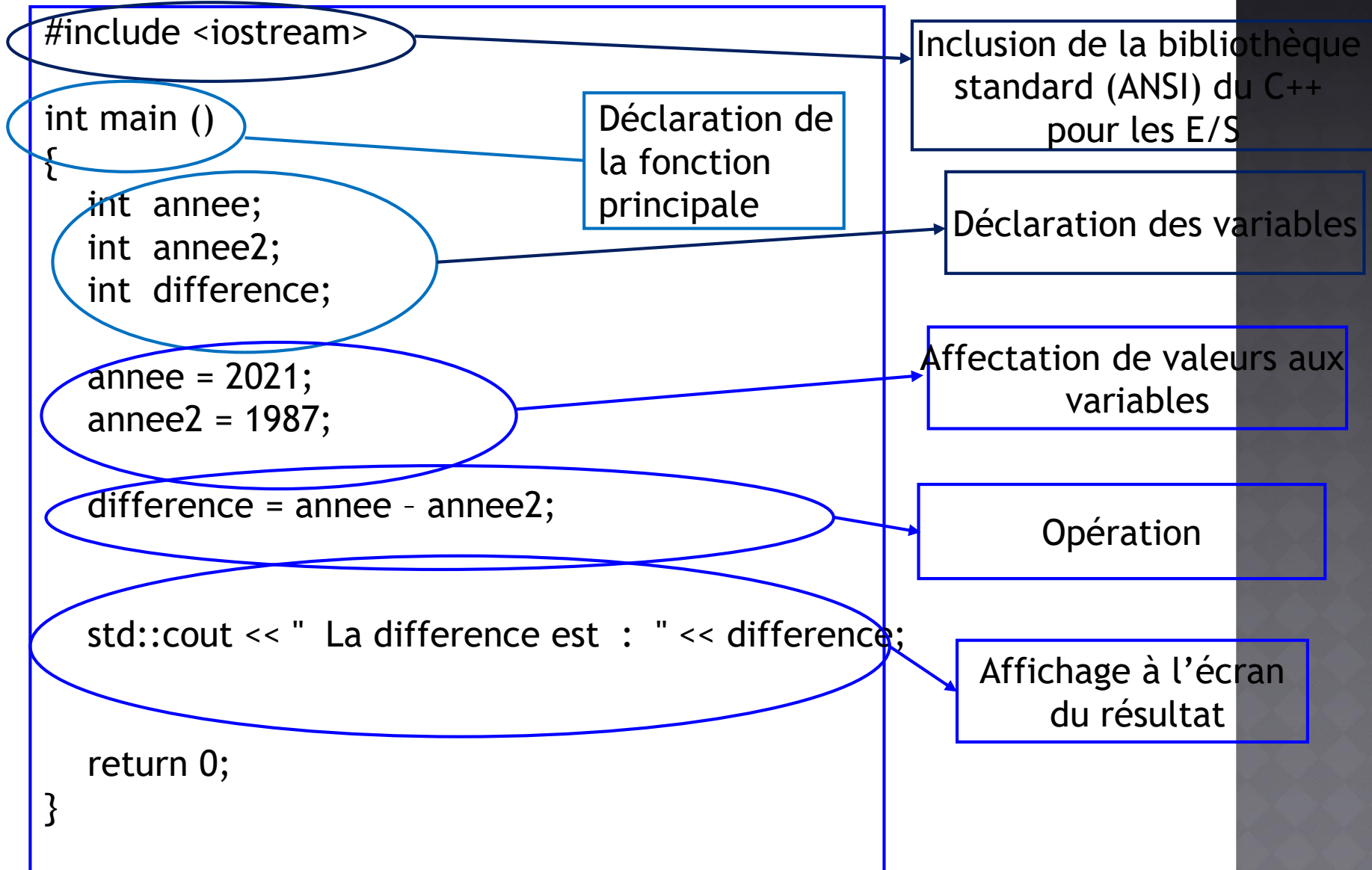
- directive de préprocesseur
 - Son rôle est de « charger » des fonctionnalités du C++ pour que le programmeur puisse effectuer certaines actions.
 - permet d'utiliser une **bibliothèque** de lecture au clavier et d'affichage de messages à l'écran dans une console

- ◉ `using namespace std;`

- **namespaces** (espaces de noms), sont des sortes de dossiers à noms.
 - Cet espace de noms est un des plus connus car il correspond à la bibliothèque standard (std), une bibliothèque livrée par défaut avec le langage C++ et dont `iostream` fait partie.
 - Généralement, cette instruction figurera à un niveau global

- ◉ Toute déclaration/instruction se termine par un **point-virgule**.

AUTRE EXEMPLE DE PROGRAMME



ETAPES DE L'EXÉCUTION D'UN PROGRAMME C++

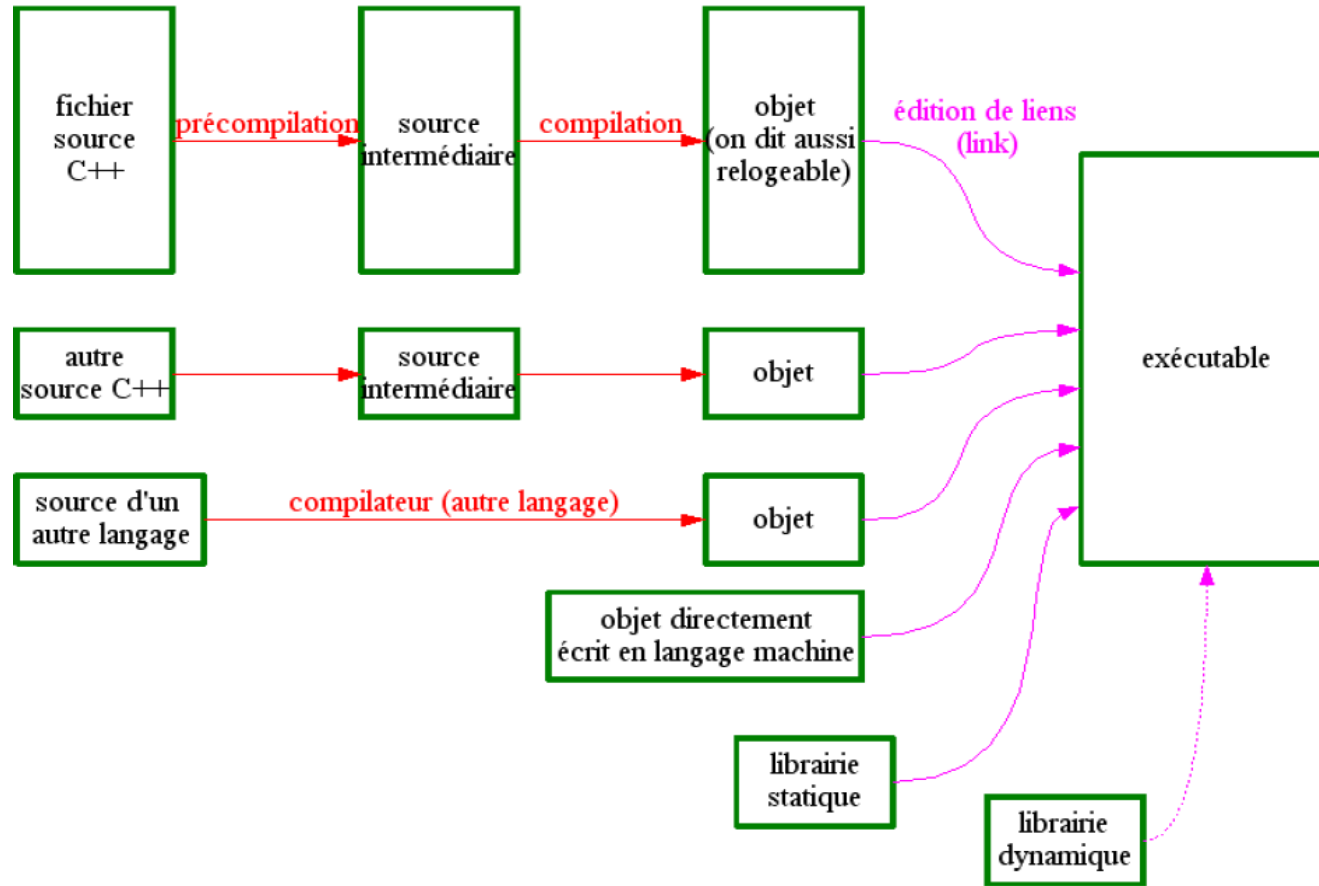
- ◎ **C++ est un langage compilé**, c'est-à-dire qu'il faut :
 - Editer le code
 - Le traduire en langage machine (c'est-à-dire en codes binaires compréhensibles par l'ordinateur) : c'est la compilation
 - L'exécuter
- ◎ La compilation d'un code source C++ se fait en plusieurs phases.
 - La première (la précompilation)
 - Elle permet des modifications du code source, et ce à l'aide des « directives » (commençant par #).
 - Deuxième (compilation)
 - Ce fichier source intermédiaire est effectivement traduit en langage machine : fichier objet (.o)

ETAPES DE L'EXÉCUTION D'UN PROGRAMME C++ (2)

◉ L'édition de liens

- Tous les fichiers objets doivent être regroupés en un seul programme exécutable : c'est le « linkage ou édition de liens ».
- Cette étape permet également de lier diverses librairies. Les librairies sont des fichiers objets fournis.
 - ◉ Exemple : fonctions graphiques, bibliothèques de fonctions mathématiques.

ETAPES DE L'EXÉCUTION D'UN PROGRAMME C++ (3)



COMPILATION SÉPARÉE

- ◎ Un programme exécutable peut être fabriqué à partir de divers éléments.
 - Fichier source
 - Contenant les définitions des fonctions
 - Fichier entête .h
 - Contenant la déclaration des fonctions
- ◎ Avantages
 - Cela permet, en cas de modification d'un seul fichier source, de ne recompiler que celui-ci (et réutiliser les autres objets tels quels).
 - Cela permet également d'utiliser un même objet dans différents programmes (s'ils nécessitent une fonction commune).
 - On peut également utiliser des objets générés à partir d'un autre langage.

LES COMMENTAIRES

- ◉ Il y a deux façons d'écrire des commentaires selon leur longueur.

```
/*Ceci est un commentaire sur plusieurs lignes
```

```
...
```

```
Fin du commentaire*/
```

```
//ceci est un commentaire sur une ligne
```


LES VARIABLES

- ◉ Tout programme informatique nécessite de stocker et de manipuler des informations. Ces informations peuvent être de plusieurs types.
- ◉ La nouveauté en C++, c'est que l'on peut désormais déclarer des variables *n'importe où* dans une fonction.
- ◉ Elle possède
 - un type (char, int...), un nom, une valeur, une taille et une portée.
- ◉ Déclaration de variable:

Nom_type nom_var; // déclaration

ou Nom_type nom_var = valeur; //déclaration et initialisation

Nom_type nom_var1, nom_var1, nom_var1, nom_var1, nom_varN;
//plusieurs variables de même type

- ◉ Déclaration locale ou globale:
 - Variables globales :
 - définies hors de toutes fonctions
 - Vues par toutes les fonctions du fichier source
 - Persistante pendant toute la durée de vie du programme
 - Variables locales:
 - Déclarées à l'intérieur d'une fonction
 - Détruites en sortie de fonction

```
int a;  
float r;  
  
int b=3;  
float s(0.25);
```

LES VARIABLES (2)

- ◉ En C++, il y a quelques règles qui régissent les différents noms autorisés ou interdits
 - les noms de variables sont constitués de lettres, de chiffres et du tiret-bas _ uniquement ;
 - le premier caractère doit être une lettre (majuscule ou minuscule) ;
 - on ne peut pas utiliser d'accents ;
 - on ne peut pas utiliser d'espaces dans le nom.
- ◉ Exemples
 - ageEtu, nom_etu ou encore PRENOM_AMI sont tous des noms valides.
 - Agebébé et _ageEtu ne sont pas des variables.
- ◉ Le langage C++ est sensible à la casse
 - Il y a une différence entre majuscules et minuscules
- ◉ Il est important d'utiliser des noms de variables qui décrivent bien ce qu'elles contiennent.

DÉFINITION DE VARIABLES CONSTANTES

◎ Constantes

- Une *constante* est un identificateur qui une fois initialisé, contient une valeur qui ne sera jamais modifiée au cours du programme.

◎ Deux syntaxes

- Utilisation du mot clé `const`
 - `int const A = 26;`
 - `float const Pi = 3.14159;`
 - `int const NFOIS = 5;`
- Utilisation de la directive **`#define`**
 - Déclaration
 - *`#define nom_constante valeur_constante`*
 - ◎ **`#define TVA 18`**



LES TYPES ÉLÉMENTAIRES

- ◉ Les entiers (1,2,-3 etc...) : **int** (4 octet)
(on peut leur ajouter les attributs: short, long, signed, unsigned)
- ◉ Les réels ou flottants (0.12, PI, ...) : **float** (4 octet)
- ◉ Les réels double précision : **double** (8 octets)
(on peut lui ajouter l'attribut : long)
- ◉ Les caractères ('a', 'b', 'c', '#', '3'...) : **char** (1 octet)
= code ASCII (0<<255) du caractère
(on peut lui ajouter les attributs signed ou unsigned)
- ◉ Le type logique ou booléen (true, false) : **bool** (1 octet)

AUTRES TYPES

- ◉ Le type string pour les chaînes de caractères
- ◉ Le type complex pour manipuler les nombres complexes.

MOTS CLÉS

- ⊙ Certains « mots-clés » sont réservés par le langage à un usage bien défini et ne peuvent pas être utilisés comme identificateurs.
- ⊙ Mots clés existants en C aussi
 - ⊙ auto double int struct break else long switch case enum register typedef char extern return union const float short unsigned continue for signed void default goto sizeof volatile do if static while

MOTS CLÉS (2)

◉ Mots clés spécifiques en C++ (par rapport au langage C)

bool	catch	class	const_cast
delete	dynamic_cast	explicit	export
false	friend	inline	mutable
namespace	new	operator	private
protected	public	reinterpret_cast	static_cast
template	this	true	throw
try	typeid	typename	using

EXEMPLES DE DÉCLARATION DE VARIABLES

```
int main() {  
    bool trouve = false;        // un booléen ne peut prendre que les valeurs true et false  
    int min(4), max(10);        // variables entières  
    int valeur = 125;           // variable entière valant 125  
    long int nombre;            // nombre est un entier long (4 octets)  
    char car = 'M';             // car est un caractère  
    unsigned int position;      // un entier non signé  
    double PI = 3.1415926535;  
    return 0;  
}
```


EXEMPLE DE TYPE DE VARIABLES SELON LA PORTÉE

```
#include <iostream>
using namespace std;
```

```
float PI = 3.14159;
```

```
int main ()
```

```
{
```

```
float rayon = 5.458;
```

```
float perimetre;
```

```
// calcul périmètre
```

```
perimetre = PI * 2* rayon;
```

```
cout << "perimetre p = " << perimetre << endl;
```

```
return 0;
```

```
}
```

Déclaration de variable globale

Déclaration de variables locales

LES OPÉRATEURS

- Affectation: =
- Opérateurs arithmétiques
 - Addition: +
 - Soustraction: -
 - Multiplication: *
 - Division: / (division entière dans le cas de **int**)
 - Modulo: % (reste de la division entière)
- Opérateurs arithmétiques unaires (un seul opérande)
 - Moins unaire: -
 - Incrémentation: ++
 - Décrément: --
- Opérateurs logiques
 - Négation: !
 - ET logique: &&
 - OU logique: ||
- Opérateurs de relation (ils retournent 1 si la condition est vraie et 0 sinon)
 - < , > , <= , >= , == , !=
- L'opérateur de succession (virgule): ,
 - Il est utilisé pour séparer les expressions.

LES OPÉRATEURS (2)

⊙ Les opérateurs composés

- += Ajouter et assigner
- -= Soustraire et assigner
- /= Diviser et assigner
- *= Multiplier et assigner
- %= Trouver le reste et assigner

■ Exemple

- **a += b; qui est la même chose que a = a+b.**
 - additionner la valeur de a et de b et mettre le résultat dans a.

LES OPÉRATEURS (3)

◎ L'opérateur *sizeof*

- Il est utilisé pour renvoyer le nombre d'octets qu'un opérande occupe dans la mémoire.
- Par exemple :
 - `int i;`
 - `char y;`
 - `i=sizeof(char);`
 - `i=sizeof(y);`
 - `cout<<"le nombre d'octet occupé est :"<<i;`
 - La sortie est: **le nombre d'octet occupé est : 1**

◎ Opérateur conditionnel :

- `?:`
- `<exp1> ? <exp2> : <exp3>`
 - `max = a>b ? a : b ;`

PRIORITÉ DES OPERATEURS

◉ Ordre décroissant

Catégorie	Opérateurs
Résolution de portée	:: (portée globale - unaire) :: (portée de classe - binaire)
Référence	() [] -> .
Unaire	+ - ++ -- ! ~ * & sizeof cast dynamic_cast static_cast reinterpret_cast const_cast new new[] delete delete[]
Sélection	->* .*
Arithmétique	* / %
Arithmétique	+ -
Décalage	<< >>
Relationnels	< <= > >=
Relationnels	== !=
Manipulation de bits	&
Manipulation de bits	^
Manipulation de bits	
Logique	&&
Logique	
Conditionnel (ternaire)	? :
Affectation	= += -= *= /= %=
	&= ^= = <<= >>=
Séquentiel	,

SAISIE ET AFFICHAGE DES DONNÉES

◉ cout (Console Output)

- L'affichage d'informations à l'écran est réalisé en envoyant des valeurs sur le « flot *cout* », comme dans :
 - `cout << n << 2*p ; // affiche les valeurs de n et de 2*p sur l'écran`

◉ cin (Console Input)

- La lecture d'informations au clavier est réalisée en extrayant des valeurs du « flot *cin* », comme dans :
- `cin >> x >> y ; // lit deux valeurs au clavier et les affecte à x et à y`

Bibliothèque
iostream

```
cout << "saisir la variable x : ";  
cin >> x;  
cout << endl;  
cout << "La valeur de x est : " << x << endl;
```

SAISIE ET AFFICHAGE DES DONNÉES (2)

```
#include <iostream>
using namespace std ;
int main()
{
    int n, p ;
    cout << "donnez une valeur pour n : " ;
    cin >> n ;
    cout << " Vous avez entré " << n << "\n" ;
    cout << "donnez une valeur pour p : " ;
    cin >> p ;
    cout << " Vous avez entré " << p << "\n" ;
    return 0;
}
```

CARACTÈRES SPÉCIAUX EN SORTIE

Notation en C++	Code ASCII	Abréviation	Signification usuelle
\a	07	BEL	cloche ou bip (alert ou audible bell)
\b	08	BS	Retour arrière (Backspace)
\f	0C	FF	Saut de page (Form Feed)
\n	0A	LF	Saut de ligne (Line Feed)
\r	0D	CR	Retour chariot (Carriage Return)
\t	09	HT	Tabulation horizontale (Horizontal Tab)
\v	0B	VT	Tabulation verticale (Vertical Tab)
\\	5C	\	
\'	2C	'	
\"	22	"	
\?	3F	?	

QUELQUES EXEMPLES

```
#include <iostream>
#include <string>
using namespace std;
int main ()
{
    string texte1 = "J'aime le chocolat";
    cout << texte1 << endl;
    string texte2 = texte1 + " sans sucre"; //concaténation
    cout << texte2 << endl;
    cout<<"Le premier caractère du texte précédent est:"<<texte2[0]<<endl;
    return 0;
}
```

QUELQUES EXEMPLES (2)

```
#include <iostream>
#include <complex>
using namespace std;
typedef complex <double> Complex;
int main ()
{
    Complex l(0,1); // ou complex <double> l(0,1); // on définit le nombre l
    cout<<"l="<<l<<endl;
    cout<<"l.l="<<l*l<<endl;
    Complex a=2.+l*4., b=3.-2.*l;
    Complex c=a+b, d=a*b;
    cout<<" a="<<a<<" b="<<b<<endl;
    cout<<" a+b="<<c<<" a*b="<<d<<endl;
    cout<<" real(a)="<<a.real()<<" imag(a)="<<a.imag()<<endl;
    return 0;
}
```

STRUCTURES DE CONTRÔLE

⊙ **Le C++ propose un certain nombre de structures de contrôle**

- Les expressions du programme qui affectent l'ordre dans lequel les instructions sont exécutées, ou qui affectent l'exécution ou non des instructions sont appelés structures de contrôle.
- Il existe deux types de structures de contrôle.
 - Les boucles et les instructions conditionnelles

LES BOUCLES

- ◉ Structure **while**
- ◉ Il y a deux utilisations possibles du while

```
while (condition)  
    instruction
```

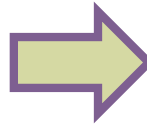
```
while (condition) {  
    instruction 1  
    ...  
    instruction n  
}
```

```
do  
    instruction  
while (condition);  
do {  
    instruction 1  
    ...  
    instruction n  
} while (condition);
```

LES BOUCLES (2)

- Structure **for**
- Le mot-clé **for** permet lui aussi d'effectuer des opérations en boucles

```
for (expr1; expr2;  
    expr3) {  
    instruction 1  
    ...  
    instruction n  
}
```



```
expr1;  
while (expr2) {  
    instruction 1  
    ...  
    instruction n  
    expr3;  
}
```

Exemple 1

```
int Tab[10];  
  
for (int i = 0; i < 10; i++)  
{  
    Tab[i] = 0;  
}
```

Exemple 2

```
int a=0;  
  
for (int x = 0, j = 10 ; x < j ; x++ , j--) //plusieurs initialisations  
                                         //et plusieurs expressions  
                                         // d'avancement de la boucle  
{  
    a++;  
    cout<<"x:"<<x<<endl;  
    cout<<"j :"<<j<<endl;  
    cout<<"a :"<<a<<endl;  
}
```

INSTRUCTIONS CONDITIONNELLES

◎ Structure de choix **if**

```
if (condition) {  
    /* si la condition est vraie */  
    instruction 1  
    ...  
    instruction n  
}
```

```
if (condition) {  
    /* si la condition est vraie */  
    instruction 1  
    ...  
    instruction n  
} else {  
    /* si la condition est fausse */  
    instruction 1  
    ...  
    instruction n  
}
```

INSTRUCTIONS CONDITIONNELLES (2)

Il est possible d'enchaîner plusieurs tests successifs grâce à **else if** :

```
if (condition1)
    /* si condition1 est vraie */
    instruction
else if (condition2)
    /* si condition1 est fausse et
condition2 est vraie */
    instruction
else
    /* si les deux conditions sont
fausses */
    instruction
```

INSTRUCTIONS CONDITIONNELLES (3)

◎ Structure de choix multiples **switch**

- permet d'effectuer un choix selon les valeurs possibles d'une expression. On sélectionne une valeur particulière en la faisant précéder du mot-clé case

INSTRUCTIONS CONDITIONNELLES (4)

◉ Structure de choix multiples **switch**

```
switch (expression) {  
    case Constante1 :  
        /* si expression est égale à Constante1 */  
        instruction 1  
        ...  
        instruction n  
        break;  
    case Constante2 :  
        /* si expression est égale à Constante2 */  
        instruction 1  
        ...  
        instruction n  
        break;  
    ...  
    default :  
        /* cas par défaut */  
        instruction 1  
        ...  
        instruction n  
        break;  
}
```

EXEMPLE

```
#include <iostream>
using namespace std ;
int main()
{
    int a,b;
    cin>>a;
    switch (a) {
        case 1 :
            b = 100;
            break;
        case 2 :
            b = 200;
            break;
        default :
            b = 0;
            break;
    }
    cout << "La valeur de b est: " << b << endl ;
    return 0;
}
```

LES INSTRUCTIONS DE BRANCHEMENT INCONDITIONNEL

◎ **break**

- L'instruction **break** a pour effet de faire sortir d'un bloc d'instructions et de passer à la première instruction située après le bloc.
- **break** est utilisé souvent avec **do while**, **for**, **while** et **switch**.

```
#include <iostream>
using namespace std ;
int main()
{
    for(int q=100; q >= 0; q-=10)
    {
        if(q == 50)
            break;
        cout<<q<<" ";
    }
    cout<<"fin du programme"<<endl;
    return 0;
}
```

100 90 80 70 60 fin du programme

LES INSTRUCTIONS DE BRANCHEMENT INCONDITIONNEL(2)

◎ **continue**

- L'instruction **continue** permet d'abandonner l'itération courante pour passer à l'itération suivante s'il y a lieu, sinon à l'instruction située immédiatement après le bloc.

```
#include <iostream>
using namespace std ;
int main()
{
    int q;
    for(q=100; q >= 0; q-=10)
    {
        if(q == 50)
            continue;
        cout<< q <<" ";
    }
    cout<<"fin du programme"<<endl;
    return 0;
}
```

100 90 80 70 60 40 30 20 10 0 fin du programme

LES INSTRUCTIONS DE BRANCHEMENT INCONDITIONNEL(3)

◎ goto

- Elle permet classiquement le branchement en un emplacement quelconque du programme.

```
#include <iostream>
using namespace std ;
int main() {
    int i ;
    for (i=1 ; i<=10 ; i++)
    {
        cout << "début tour " << i << "\n" ;
        cout << "bonjour\n" ;
        if (i==3)
            goto sortie ;
        cout << "fin tour " << i << "\n" ;
    }
    cout<<"Je teste un programme"<<endl;
    sortie : cout << "après la boucle" ;
    return 0 ;
}
```

```
début tour 1
bonjour
fin tour 1
début tour 2
bonjour
fin tour 2
début tour 3
bonjour
après la boucle
```

TRANSTYPAGE

- ⊙ Il est possible de forcer la conversion d'une expression quelconque dans un type de son choix, grâce à l'opérateur dit de « cast ».
- ⊙ Par exemple, si `n` et `p` sont des variables entières, l'expression :
 - `(double) n/p;`
 - ⊙ aura comme valeur celle de l'expression entière `n/p` convertie en double.
- ⊙ Le langage C++ possède ses mots-clefs propres à lui qui permettent de convertir des variables ou des objets.
 - Les conversions d'un type numérique vers un autre type numérique sont qualifiées par `static_cast`
 - ⊙ ce qui signifie qu'elles sont quasi indépendantes de l'implémentation.
 - ⊙ `(double) n/p` correspond alors à `static_cast<double>(n)/p`
//attention les parenthèses sont obligatoires
- ⊙ Il existe d'autres façons de qualifier un opérateur de cast : `const_cast`, `reinterpret_cast` et `dynamic_cast`.

INSTALLATION ET PRISE EN MAIN DE DEV-C++

- ◎ Télécharger le fichier .exe de Dev-C++ et l'installer.
- ◎ Installation sous linux du package g++-8

TD1

⊙ Exercice 1

Soient les déclarations suivantes :

```
int n = 5, p = 9 ;
```

```
int q ;
```

```
float x ;
```

Quelle est la valeur affectée aux différentes variables concernées par chacune des instructions suivantes ?

- ⊙ `q = n < p ; /* 1 */`
- ⊙ `q = n == p ; /* 2 */`
- ⊙ `q = p % n + p > n ; /* 3 */`
- ⊙ `x = p / n ; /* 4 */`
- ⊙ `x = (float) p / n ; /* 5 */`
- ⊙ `x = (p + 0.5) / n ; /* 6 */`
- ⊙ `x = (int) (p + 0.5) / n ; /* 7 */`
- ⊙ `q = n * (p > n ? n : p) ; /* 8 */`
- ⊙ `q = n * (p < n ? n : p) ; /* 9 */`

TD1

⊙ Exercice 2

Quels résultats fournira ce programme :

```
#include <iostream>
using namespace std ;
int main(){
int n, p, q ;
n = 5 ; p = 2 ; /* cas 1 */
q = n++ > p || p++ != 3 ;
cout << "A : n = " << n << " p = " << p << " q = " << q << "\n" ;
n = 5 ; p = 2 ; /* cas 2 */
q = n++ < p || p++ != 3 ;
cout << "B : n = " << n << " p = " << p << " q = " << q << "\n" ;
n = 5 ; p = 2 ; /* cas 3 */
q = ++n == 3 && ++p == 3 ;
cout << "C : n = " << n << " p = " << p << " q = " << q << "\n" ;
n = 5 ; p = 2 ; /* cas 4 */
q = ++n == 6 && ++p == 3 ;
cout << "D : n = " << n << " p = " << p << " q = " << q << "\n" ;
return 0;
}
```

⦿ Exercice 3

Quels résultats fournira ce programme :

```
int main(){
int n=0 ;
do {
    if (n%2==0) {
        cout << n << " est pair\n" ;
        n += 3 ;
        continue ;
    }
    if (n%3==0) {
        cout << n << " est multiple de 3\n" ;
        n += 5 ;
    }
    if (n%5==0) {
        cout << n << " est multiple de 5\n" ;
        break ;
    }
    n += 1 ;
} while (1) ;
return 0 ;
}
```

⦿ Exercice 4

Quelle est la sortie du programme suivant ?

```
#include <iostream>
using namespace std ;
int main(){
int i, n ;
for (i=0, n=0 ; i<5 ; i++)
    n++ ;
cout << "A : i = " << i << " n = " << n << "\n" ;
for (i=0, n=0 ; i<5 ; i++, n++) {}
cout << "B : i = " << i << " n = " << n << "\n" ;
for (i=0, n=50 ; n>10 ; i++, n-= i ) {}
cout << "C : i = " << i << " n = " << n << "\n" ;
for (i=0, n=0 ; i<3 ; i++, n+=i, cout << "D : i = " << i << " n = " <<
n << "\n" ) ;
cout << "E : i = " << i << " n = " << n << "\n" ;
return 0;
}
```

TD1

◉ Exercice 5

Ecrire un programme de facturation avec remise. Il lit en donnée un simple prix hors taxes et calcule le prix TTC correspondant (avec un taux de TVA constant de 18 %). Il établit ensuite une remise dont le taux dépend de la valeur ainsi obtenue, à savoir :

- 0 % pour un montant inférieur à 1 000 euros ;
- 1 % pour un montant supérieur ou égal à 1 000 euros et inférieur à 2 000 euros ;
- 3 % pour un montant supérieur ou égal à 2 000 euros et inférieur à 5 000 euros ;
- 5 % pour un montant supérieur ou égal à 5 000 euros.

BIBLIOGRAPHIE

- ◉ Comment programmer en C++ de H. Deitel
- ◉ Le langage C++ de B. Stroustrup
- ◉ C / C++ / C# - La bible du programmeur de K. Jamsa
- ◉ Programmer en langage C++ de Claude Delannoy